

AUTOMATION PORTFOLIO — TECHNICAL RECORD

SOPHON FINANCE SYSTEMS

AI-Driven Finance & Accounting Automation | github.com/sophonfinance-wq/finance-automation-portfolio

FINANCE OPERATIONS ATLAS

A deterministic, single-file documentation generator that renders a fictional finance department's operating map — drives, workstreams, a location directory, and a recurring calendar — from a typed Python data model into one self-contained, dependency-free interactive HTML artifact.

Report No.	SFS-ENG-008	Package	finance-atlas
System	Finance Operations Atlas	Data	Fictional / seeded
Run	py generate.py	Status	Public — portfolio record

1.0 OVERVIEW AND SCENARIO

A new analyst joining the finance and accounting group of Demo Holdings Inc. — a fictional real-estate investment company with roughly 120 seeded legal entities across three regions (Northwest, Southwest, Central) and a December 31 fiscal year-end — needs to learn where records live and how each recurring process runs without shadowing a colleague for weeks. They open the single generated HTML file, type "t1134" into the Find It directory to jump to the cross-border Surplus & ACB working area, and step through the Month-End Cash Reconciliation pipeline to see its materiality and dormant-account rules. Because the repository is public, the system also guards against a maintainer accidentally pasting a real bank or entity name into atlas_data.py: the confidentiality deny-list sweep fails the build and reports only a SHA-256 digest and the affected surface, pinpointing the leak without republishing the term in CI logs.

2.0 WHAT THE SYSTEM AUTOMATES

- Renders the entire department map — 5 drives, 14 folder briefings, 4 workstream pipelines, a 33-row Find It directory, and a 16-event calendar — from a single data model (atlas_data.py) into one self-contained HTML file via `py generate.py`.
- Resolves palette keys to hex, flattens typed dataclasses to JSON-ready structures, and serializes the model to compact ASCII-safe JSON embedded in window.ATLAS, then appends a dependency-free client script that builds all five interactive views browser-side.
- Substitutes palette tokens into CSS :root variables so the stylesheet, chips, and legend all bind to the same color source.
- Builds the live-filtered Find It location table that narrows results as the user types and updates an aria-live result counter.
- Escapes every piece of dynamic and metadata text (server-side html.escape plus a client-side esc()) and neutralizes any '</' in the JSON payload so data can never break the page or close the script early.
- Sweeps six published surfaces (both source modules, the README, the generated artifact, and two recursive tree/repo sweeps) against a confidentiality deny list, reporting only the digest and surface on a hit.

- Prints a build summary on each run — drive, folder-briefing, workstream, step, Find It, and calendar counts plus the output byte size.

3.0 OPERATING PROCEDURE

- 1 A maintainer authors or edits the department map as frozen dataclasses (Drive, Folder, Workstream, Step) and plain dicts in `atlas_data.py`, where all display text lives.
- 2 ``py generate.py`` imports the model and calls `build_payload()`, resolving each drive's and workstream's palette key to a hex value and flattening the dataclasses to ordered, JSON-ready dictionaries.
- 3 `payload_json()` serializes the payload to compact ASCII JSON and replaces every `'</'` with `'<\'` so the embedded data cannot terminate the script block early.
- 4 A `string.Template` substitutes the palette dictionary into the CSS, binding each token to a `--var` in `:root`.
- 5 `render()` assembles the HTML shell, escaping title/description/hero/footer metadata, embeds the JSON payload in a first `<script>`, and appends the client application script as a second `<script>`.
- 6 The document is encoded as UTF-8 and written to `out/finance-operations-atlas.html` (parent directories created automatically; `--out` overrides the path).
- 7 Opened in any browser, the client script reads `window.ATLAS` and constructs the Overview, Drive Map, Workstreams, Find It, and Calendar views entirely client-side with no server, build chain, or external resource.
- 8 CI runs the pytest suite — data-model integrity, artifact structure/accessibility, generator determinism, and the confidentiality deny-list sweep — to gate the committed artifact and sources.

4.0 CONTROLS AND SAFEGUARDS

- Determinism: the generator uses no timestamps and no randomness, so two runs produce byte-identical output; a test re-renders in-process and compares against the committed artifact for freshness.
- Single source of truth / read-only rendering: all display text lives in `atlas_data.py` and the generator only reads the model and writes one output file, never mutating inputs.
- Confidentiality deny-list linter: 85 one-way SHA-256 digests are matched (case-insensitive, word-boundary-aware, via token n-grams) across six surfaces; failures print only the digest and surface, never the matched plaintext, so a leak is caught without being reprinted.
- Fictional-only enforcement: tests assert every Find It location begins with a fictional drive prefix (`G:/W:/L:/R/`), the title is marked 'demonstration', and the footer carries the fictional disclaimer.
- Injection-safe output: server-side `html.escape` on metadata plus a client-side `esc()` on all dynamic text, no inline event handlers, exactly two inline scripts, and assertions that no external `src/href/URLs` or external-resource tags exist.
- Per-record completeness gating: every folder briefing must carry at least one key location and one working note, every workstream at least three steps, and every step at least one input/output row — checked individually so one gap fails exactly one test.

5.0 VERIFICATION AND TESTING

Correctness is proven by a pytest suite of 928 collected tests across four modules that check the data model per record, compare the generated artifact's embedded JSON payload against the model field by field, and verify document structure, self-containment, and accessibility (balanced tags, unique ids, no external URLs, labelled search, reduced-motion and focus styles). Determinism is proven directly: two generator runs are

asserted byte-identical and the committed artifact is compared to a fresh in-process render. A separate confidentiality module runs an 85-digest deny list across six surfaces, with matcher self-tests confirming it detects a seeded canary case-insensitively yet respects word boundaries. In this Windows clone 927 of 928 tests pass; the sole exception is the committed-artifact freshness check, tripped by CRLF line-ending normalization on checkout rather than any content difference.

6.0 KEY FACTS

Attribute	Value
Automated tests (pytest, 4 modules)	928 collected
Confidentiality deny list	85 SHA-256 digests swept over 6 surfaces
Drives and folder briefings	5 drives, 14 briefings
Workstreams and steps	4 pipelines, 24 steps
Find It location directory	33 rows
Recurring calendar events	16 events

This record documents a self-contained system in the Sophon Finance Systems automation portfolio. All data is fictional and seeded; no employer or client workpaper, entity, methodology, path, or figure is reproduced. Prepared 14 July 2026.